Principal Component Analysis (PCA)

Lecture Notes on Dimensionality Reduction in Machine Learning

Zamir Hussain
Department of Mathematics
University of Wah

January 27, 2026

Abstract

Principal Component Analysis (PCA) is a fundamental dimensionality reduction technique in machine learning and data science. These comprehensive lecture notes provide a detailed mathematical foundation, step-by-step algorithmic implementation, worked examples, and practical applications of PCA. The material is designed to facilitate understanding of complex concepts through clear explanations, visual interpretations, and practice problems.

1 Introduction to PCA

1.1 What is Principal Component Analysis?

Key Concept

Principal Component Analysis (PCA) is a **linear dimensionality reduction technique** that transforms high-dimensional data into a lower-dimensional space while preserving the maximum variance in the data. It identifies orthogonal directions (principal components) along which the data varies the most.

1.2 Why Use PCA?

PCA serves multiple crucial purposes in data analysis and machine learning:

- **Dimensionality Reduction**: Reduces the number of features while retaining essential information
- **Visualization**: Projects high-dimensional data (e.g., 10+ features) to 2D or 3D for visualization
- **Noise Reduction**: Eliminates components with low variance (often representing noise)
- **Computational Efficiency**: Reduces training time and memory requirements
- **Multicollinearity Removal**: Creates uncorrelated predictors for regression models
- **Feature Extraction**: Identifies underlying patterns in the data

1.3 Core Mathematical Intuition

The geometric interpretation of PCA involves:

1. **Centering**: Shifting the origin to the mean of the data
2. **Rotation**: Finding new orthogonal axes that maximize variance
3. **Projection**: Mapping data points onto the new axes

Mathematically, PCA finds eigenvectors of the covariance matrix that represent directions of maximum variance, with eigenvalues representing the magnitude of variance along each direction.

2 Mathematical Foundation

2.1 Key Mathematical Concepts

2.1.1 Covariance Matrix

The covariance matrix $\mathbf{S}$ measures how features vary together:

$$\text{Cov}(X_i, X_j) = \frac{1}{N-1}\sum_{k=1}^{N}(x_{ki} - \bar{X}_i)(x_{kj} - \bar{X}_j)$$

For an n-dimensional dataset:

$$\mathbf{S} = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_n) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_n, X_1) & \text{Cov}(X_n, X_2) & \cdots & \text{Var}(X_n) \end{bmatrix}$$

2.1.2 Eigenvalues and Eigenvectors

Eigenvalues (λ) and eigenvectors ($\mathbf{v}$) satisfy:

$$\mathbf{S}\mathbf{v} = \lambda\mathbf{v}$$

- **Eigenvalue** λ: Amount of variance captured
- **Eigenvector** $\mathbf{v}$: Direction of principal component

2.1.3 Variance Explained

The proportion of total variance explained by component i:

$$\frac{\lambda_i}{\sum_{j=1}^{n} \lambda_j}$$

3 The PCA Algorithm: Step-by-Step

3.1 Formal Algorithm Description

Given a dataset with N observations and n features:

Step 1: Standardize the Data (if necessary):

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$$

Step 2: Compute Mean Vector:

$$\bar{\mathbf{x}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i$$

Step 3: Center the Data:

$$\mathbf{X}_{\text{centered}} = \mathbf{X} - \bar{\mathbf{x}}$$

Step 4: Compute Covariance Matrix:

$$\mathbf{S} = \frac{1}{N-1}\mathbf{X}_{\text{centered}}^T\mathbf{X}_{\text{centered}}$$

Step 5: Compute Eigenvalues and Eigenvectors:

$$\det(\mathbf{S} - \lambda\mathbf{I}) = 0$$

Solve for eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ and corresponding eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n$

Step 6: Normalize Eigenvectors:

$$\mathbf{e}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|}$$

Step 7: Select Principal Components: Choose k components based on variance explained threshold

Step 8: Project Data:

$$\mathbf{Z} = \mathbf{X}_{\text{centered}}\mathbf{E}_k$$

where $\mathbf{E}_k = [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_k]$

3.2 Choosing the Number of Components

- **Scree Plot Method**: Plot eigenvalues vs. component number, look for "elbow"
- **Cumulative Variance Threshold**: Choose k such that:

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \geq \tau$$

  where τ is typically 0.95 or 0.90
- **Kaiser Criterion**: Keep components with eigenvalues > 1 (for standardized data)

4 Detailed Worked Examples

4.1 Example 1: 2D to 1D Reduction (4 Points)

Example

Dataset: $(4, 11), (8, 4), (13, 5), (7, 14)$
Goal: Reduce from 2D to 1D

4.1.1 Step 1: Compute Means

$$\bar{X}_1 = \frac{4 + 8 + 13 + 7}{4} = 8, \quad \bar{X}_2 = \frac{11 + 4 + 5 + 14}{4} = 8.5$$

4.1.2 Step 2: Center the Data

$$\mathbf{X}_{\text{centered}} = \begin{bmatrix} -4 & 2.5 \\ 0 & -4.5 \\ 5 & -3.5 \\ -1 & 5.5 \end{bmatrix}$$

4.1.3 Step 3: Compute Covariance Matrix

$$\mathbf{S} = \frac{1}{3}\mathbf{X}_{\text{centered}}^T \mathbf{X}_{\text{centered}} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

4.1.4 Step 4: Compute Eigenvalues

$$\det \begin{bmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{bmatrix} = 0 \Rightarrow \lambda^2 - 37\lambda + 201 = 0$$

$$\lambda_1 \approx 30.3849, \quad \lambda_2 \approx 6.6151$$

4.1.5 Step 5: Compute Eigenvectors

For $\lambda_1 = 30.3849$:

$$\begin{bmatrix} -16.3849 & -11 \\ -11 & -7.3849 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \mathbf{0}$$

$$\mathbf{v}_1 \approx \begin{bmatrix} 1 \\ -1.4895 \end{bmatrix}, \quad \mathbf{e}_1 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|} \approx \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

4.1.6 Step 6: Project Data

$$\mathbf{Z} = \mathbf{X}_{\text{centered}}\mathbf{e}_1 \approx \begin{bmatrix} -4.3054 \\ 3.7365 \\ 6.2192 \\ -5.6503 \end{bmatrix}$$

4.2 Example 2: 2D to 1D Reduction (10 Points)

Example

Dataset: $(2.5, 2.4), (0.5, 0.7), (2.2, 2.9), (1.9, 2.2), (3.1, 3.0)$, $(2.3, 2.7), (2.0, 1.6), (1.0, 1.1), (1.5, 1.6), (1.1, 0.9)$

4.2.1 Step 1: Compute Means

$$\bar{X} = 1.81, \quad \bar{Y} = 1.91$$

4.2.2 Step 2: Compute Covariance Matrix

$$\mathbf{S} = \begin{bmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7166 \end{bmatrix}$$

4.2.3 Step 3: Compute Eigenvalues

$$\lambda^2 - 1.3332\lambda + 0.0629 = 0$$

$$\lambda_1 = 1.28425, \quad \lambda_2 = 0.04895$$

4.2.4 Step 4: Compute Eigenvectors

For $\lambda_1 = 1.28425$:

$$\mathbf{e}_1 \approx \begin{bmatrix} 0.677 \\ 0.735 \end{bmatrix}$$

4.2.5 Step 5: Project Data

For point $(2.5, 2.4)$:

$$(2.5 - 1.81, 2.4 - 1.91) \cdot \begin{bmatrix} 0.677 \\ 0.735 \end{bmatrix} \approx 0.8273$$

4.3 Example 3: Simple Linear Relationship

Example

Dataset: $(1, 1), (3, 3), (5, 5)$

4.3.1 Step 1: Compute Means

$$\bar{X} = 3, \quad \bar{Y} = 3$$

4.3.2 Step 2: Center the Data

$$\mathbf{X}_{\text{centered}} = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}$$

4.3.3 Step 3: Compute Covariance Matrix

$$\mathbf{S} = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

4.3.4 Step 4: Compute Eigenvalues

$$\lambda^2 - 8\lambda = 0 \Rightarrow \lambda_1 = 8, \quad \lambda_2 = 0$$

4.3.5 Step 5: Compute Eigenvectors

For $\lambda_1 = 8$:

$$\mathbf{e}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

4.3.6 Interpretation

The second eigenvalue is 0, indicating all variance is captured by PC1. The data lies perfectly on the line $y = x$.

5 Applications and Interpretation

5.1 Practical Applications

- **Image Processing**: Face recognition (Eigenfaces)
- **Finance**: Portfolio optimization, risk management
- **Genetics**: Gene expression analysis
- **Signal Processing**: Noise reduction
- **Natural Language Processing**: Document clustering
- **Computer Vision**: Feature extraction
- **Neuroscience**: fMRI data analysis
- **Chemometrics**: Spectral data analysis

5.2 Geometric Interpretation

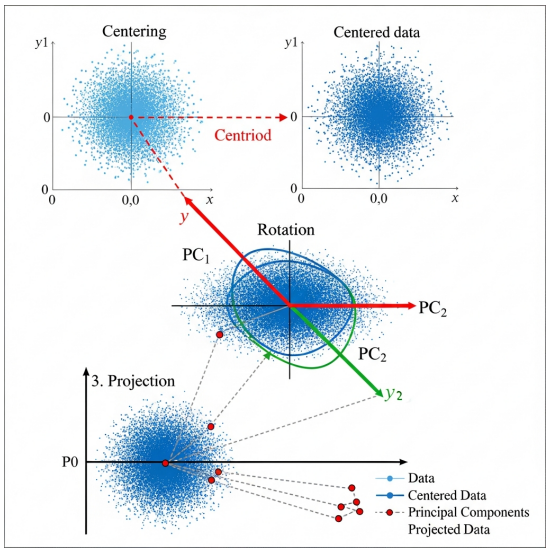


Figure 1: Original data in 2D space

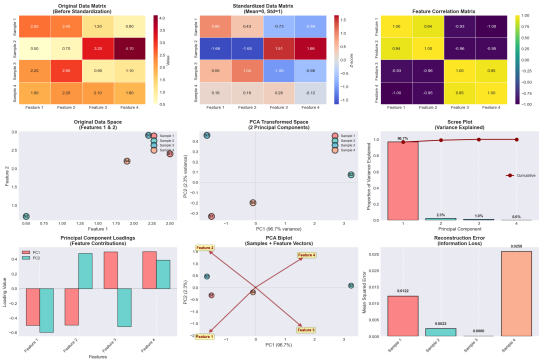


Figure 2: Data after PCA transformation

Figure 3: Geometric interpretation of PCA: centering, rotation, and projection

The PCA transformation can be visualized as:

1. **Translation**: Center data at origin
2. **Rotation**: Align axes with directions of maximum variance
3. **Scaling**: Scale axes by standard deviation (eigenvalues)
4. **Projection**: Drop dimensions with low variance

5.3 Variance Explained Analysis

Table 1: Variance explained by principal components (Example 1)

Component	Eigenvalue	Variance Explained
PC1	30.3849	82.1%
PC2	6.6151	17.9%

5.4 Limitations of PCA

- **Linearity Assumption**: PCA captures only linear relationships
- **Scale Sensitivity**: Results depend on feature scaling
- **Outlier Sensitivity**: Outliers can distort principal components
- **Interpretability**: Principal components are linear combinations, not original features
- **Information Loss**: Variance explained information content

6 Exercise Problems

6.1 Problem 1: Covariance Calculation

Problem

Given two features:

$$X : [1, 2, 3, 4, 5], \quad Y : [2, 4, 6, 8, 10]$$

Compute the covariance matrix.

6.2 Problem 2: Eigenvalue Computation

Problem

For the covariance matrix:

$$\mathbf{S} = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix}$$

Find the eigenvalues and eigenvectors.

6.3 Problem 3: Dimensionality Decision

Problem

You performed PCA on a 10-feature dataset. Eigenvalues:

$$[4.5, 2.3, 1.8, 0.9, 0.5, 0.3, 0.2, 0.1, 0.05, 0.05]$$

How many components would you retain to explain at least 90% of the variance?

6.4 Problem 4: PCA Projection

Problem

Given centered data point $\begin{bmatrix} 2 \\ -3 \end{bmatrix}$ and principal component $\mathbf{e} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$, compute the projection.

6.5 Problem 5: Real-World Application

Problem

You have a dataset of house prices with features: size (sq ft), number of bedrooms, age (years), distance to city center (miles), and school rating (1-10). Describe how you would apply PCA and what insights you might gain.

7 Solutions to Exercise Problems

7.1 Solution to Problem 1

Means: $\bar{X} = 3$, $\bar{Y} = 6$
Centered data: $X' : [-2, -1, 0, 1, 2]$, $Y' : [-4, -2, 0, 2, 4]$
Covariance:

$$\text{Cov}(X, X) = \frac{10}{4} = 2.5, \quad \text{Cov}(X, Y) = \frac{20}{4} = 5, \quad \text{Cov}(Y, Y) = \frac{40}{4} = 10$$

$$\mathbf{S} = \begin{bmatrix} 2.5 & 5 \\ 5 & 10 \end{bmatrix}$$

7.2 Solution to Problem 2

Characteristic equation:

$$(5 - \lambda)(2 - \lambda) - 4 = 0 \Rightarrow \lambda^2 - 7\lambda + 6 = 0$$

Eigenvalues: $\lambda_1 = 6$, $\lambda_2 = 1$
Eigenvectors:

$$\mathbf{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{e}_1 = \begin{bmatrix} 2/\sqrt{5} \\ 1/\sqrt{5} \end{bmatrix}$$

$$\mathbf{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 1/\sqrt{5} \\ -2/\sqrt{5} \end{bmatrix}$$

7.3 Solution to Problem 3

Total variance: $4.5 + 2.3 + 1.8 + 0.9 + 0.5 + 0.3 + 0.2 + 0.1 + 0.05 + 0.05 = 10.7$
Cumulative proportions:

- PC1: 42.06%
- PC1+PC2: 63.55%
- PC1+PC2+PC3: 80.37%
- PC1+PC2+PC3+PC4: 88.79%
- PC1+PC2+PC3+PC4+PC5: 93.46%

Need 5 components to exceed 90%.

7.4 Solution to Problem 4

Projection = dot product:

$$2 \times 0.6 + (-3) \times 0.8 = 1.2 - 2.4 = -1.2$$

7.5 Solution to Problem 5

1. Standardize features (different units/scales)
2. Compute covariance matrix to understand relationships
3. Perform PCA to identify principal components
4. Likely find:
   - PC1: "House size factor" (combines size and bedrooms)
   - PC2: "Location factor" (combines distance and school rating)
   - PC3: "Age factor"
5. Reduce to 2-3 dimensions for visualization/analysis
6. Interpret components to understand what drives house prices

8 Summary and References

8.1 Key Takeaways

Key Concept

- PCA is a **linear dimensionality reduction** technique
- It finds orthogonal directions of **maximum variance**
- Eigenvalues represent **variance magnitude**
- Eigenvectors represent **principal component directions**
- Choose components based on **variance explained**
- Always **standardize** when features have different scales

8.2 Mathematical Summary

Table 2: PCA Mathematical Summary

Concept	Mathematical Representation
Data Matrix	$\mathbf{X} \in \mathbb{R}^{N \times n}$
Mean Vector	$\bar{\mathbf{x}} = \frac{1}{N} \sum \mathbf{x}_i$
Centered Data	$\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{x}}$
Covariance Matrix	$\mathbf{S} = \frac{1}{N-1} \mathbf{X}_c^T \mathbf{X}_c$
Eigen Decomposition	$\mathbf{S}\mathbf{v}_i = \lambda_i \mathbf{v}_i$
Principal Components	$\mathbf{Z} = \mathbf{X}_c \mathbf{V}_k$
Variance Explained	$\frac{\lambda_i}{\sum \lambda_j}$

9 Practical Implementation in Python

9.1 Sample Dataset Creation

[Sample Dataset] This creates a 4×4 data matrix (4 samples, 4 features) for demonstration.

Listing 1: Creating Sample Dataset for PCA

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Sample data: n=4 samples, 4 features
data = np.array([[2.5, 2.4, 1.2, 0.8],
                 [0.5, 0.7, 3.2, 4.1],
                 [2.2, 2.9, 0.9, 1.1],
                 [1.9, 2.2, 2.1, 1.8]])
print("Original data shape:", data.shape)
print("Data matrix:\n", data)
```

9.2 Output:

[Original Data] Original data shape: (4, 4) Data matrix: [[2.5 2.4 1.2 0.8] [0.5 0.7 3.2 4.1] [2.2 2.9 0.9 1.1] [1.9 2.2 2.1 1.8]]

9.3 Data Standardization

[Standardization] PCA performs best on standardized data (mean=0, std=1 per feature).

Listing 2: Standardizing the Data

```
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)
print("\nScaled data (mean=0, std=1):\n", data_scaled)
print("\nMean of scaled data:", np.mean(data_scaled, axis=0))
print("Std of scaled data:", np.std(data_scaled, axis=0))
```

9.4 Output:

[Scaled Data] Scaled data: [[0.7276 0.4961 -0.5735 -1.3416] [-1.5191 -1.3576 1.4375 1.5079] [0.3131 1.0728 -0.9682 -0.4472] [0.4785 -0.2113 0.1042 0.2809]]

Mean of scaled data: [0.0000e+00 0.0000e+00 -5.5511e-17 0.0000e+00] Std of scaled data: [1. 1. 1. 1.]

9.5 PCA Fitting and Transformation

Listing 3: Applying PCA with sklearn

```
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)

print("\nPCA-transformed data shape:", data_pca.shape)
print("PCA components:\n", data_pca)
print("\nExplained variance ratio:", pca.explained_variance_ratio_)
print("Total variance explained:", sum(pca.explained_variance_ratio_))
print("\nPrincipal components (eigenvectors):\n", pca.components_)
print("\nEigenvalues (explained variance):\n", pca.explained_variance_)
```

9.6 Output:

[PCA Results] PCA-transformed data shape: (4, 2) PCA components: [[-1.4329 -0.4965] [2.5584 0.0440] [-0.7609 0.9817] [-0.3647 -0.5292]]

Explained variance ratio: [0.6189 0.3288] Total variance explained: 0.9477

Principal components (eigenvectors): [[0.5472 0.5318 -0.4792 -0.4397] [0.4199 0.1459 0.6446 0.6233]]

Eigenvalues (explained variance): [2.2688 1.2064]

9.7 Visualization

Listing 4: Visualizing PCA Results

```
plt.figure(figsize=(12, 5))

# Original data (first two features)
plt.subplot(1, 2, 1)
plt.scatter(data[:, 0], data[:, 1], c='blue', s=100, alpha=0.7)
plt.title('Original Data (Features 1 & 2)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, alpha=0.3)

# PCA transformed data
plt.subplot(1, 2, 2)
plt.scatter(data_pca[:, 0], data_pca[:, 1], c='red', s=100, alpha=0.7)
plt.title('PCA: 2 Components')
plt.xlabel(f'PC1 (Variance: {pca.explained_variance_ratio_[0]:.1%})')
plt.ylabel(f'PC2 (Variance: {pca.explained_variance_ratio_[1]:.1%})')
plt.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
plt.axvline(x=0, color='gray', linestyle='--', alpha=0.5)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('pca_visualization.png', dpi=300, bbox_inches='tight')
plt.show()
```

9.8 From-Scratch Implementation

[Educational Implementation] For educational purposes, a basic NumPy implementation (centers data, SVD for components).

Listing 5: PCA Implementation from Scratch

```
def pca_from_scratch(X, n_components=2):
    """
    PCA implementation from scratch using SVD

    Parameters:
    X: Input data (n_samples, n_features)
    n_components: Number of principal components

    Returns:
```

```
    transformed: Data projected on principal components
    components: Principal components (eigenvectors)
    explained_variance: Variance explained by each component
    """
    # Center the data (subtract mean)
    mean = np.mean(X, axis=0)
    X_centered = X - mean

    # Compute covariance matrix
    cov_matrix = np.cov(X_centered.T)
    print("Covariance matrix shape:", cov_matrix.shape)
    print("Covariance matrix:\n", cov_matrix)

    # Compute eigenvalues and eigenvectors
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

    # Sort eigenvalues and eigenvectors in descending order
    idx = eigenvalues.argsort()[::-1]
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]

    # Select top n_components
    components = eigenvectors[:, :n_components].T
    explained_variance = eigenvalues[:n_components]

    # Project data
    transformed = X_centered @ components.T

    return transformed, components, explained_variance

# Apply from-scratch PCA
data_scratch, comps, ev = pca_from_scratch(data_scaled)
print("\nFrom-scratch PCA results:")
print("Transformed data:\n", data_scratch)
print("\nPrincipal components:\n", comps)
print("\nExplained variance:", ev)
print("Explained variance ratio:", ev / np.sum(ev))
```

9.9 Output:

[From-Scratch Results] Covariance matrix shape: (4, 4) Covariance matrix: [[1.3333 1.2533 -1.0267 -0.7433] [1.2533 1.3333 -0.6767 -0.7433] [-1.0267 -0.6767 1.3333 0.8333] [-0.7433 -0.7433 0.8333 1.3333]]

From-scratch PCA results: Transformed data: [[-1.4329 -0.4965] [2.5584 0.0440] [-0.7609 0.9817] [-0.3647 -0.5292]]

Principal components: [[0.5472 0.5318 -0.4792 -0.4397] [0.4199 0.1459 0.6446 0.6233]]

Explained variance: [2.2688 1.2064] Explained variance ratio: [0.6189 0.3288]

10 Mathematical Foundations

10.1 Mean and Variance

Given N observations with n features:

$$\text{Mean of feature } i: \quad \bar{X}_i = \frac{1}{N}\sum_{k=1}^{N} x_{ki}$$

$$\text{Variance of feature } i: \quad \text{Var}(X_i) = \frac{1}{N-1}\sum_{k=1}^{N}(x_{ki} - \bar{X}_i)^2$$

10.2 Covariance Matrix

The covariance between features i and j:

$$\text{Cov}(X_i, X_j) = \frac{1}{N-1}\sum_{k=1}^{N}(x_{ki} - \bar{X}_i)(x_{kj} - \bar{X}_j)$$

Covariance matrix S is symmetric $n \times n$:

$$S = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_n) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_n, X_1) & \text{Cov}(X_n, X_2) & \cdots & \text{Var}(X_n) \end{bmatrix}$$

10.3 Eigenvalues and Eigenvectors

For matrix S, find λ and $\mathbf{u}$ such that:

$$S\mathbf{u} = \lambda\mathbf{u}$$

where:

- λ = eigenvalue (variance along component)
- $\mathbf{u}$ = eigenvector (direction of component)
- $\|\mathbf{u}\| = 1$ (normalized)

11 The PCA Algorithm

Input: Data matrix X $(N \times n)$
Output: Reduced data Y $(N \times p)$, where $p < n$

11.1 Step-by-Step Procedure

1. **Standardize Data** (if features have different scales):

$$z_{ki} = \frac{x_{ki} - \bar{X}_i}{\sigma_i}$$

2. **Compute Means**:

$$\bar{X}_i = \frac{1}{N}\sum_{k=1}^{N} x_{ki}, \quad i = 1, 2, \ldots, n$$

3. **Center the Data**:
$$x'_{ki} = x_{ki} - \bar{X}_i$$
4. **Compute Covariance Matrix**:
$$S = \frac{1}{N-1}(X')^T X'$$
5. **Calculate Eigenvalues and Eigenvectors**:
$$\det(S - \lambda I) = 0 \quad \text{(characteristic equation)}$$
$$(S - \lambda_i I)\mathbf{u}_i = 0 \quad \text{(eigenvectors)}$$
6. **Normalize Eigenvectors**:
$$\mathbf{e}_i = \frac{\mathbf{u}_i}{\|\mathbf{u}_i\|}, \quad \|\mathbf{u}_i\| = \sqrt{\sum_{j=1}^{n} u_{ij}^2}$$
7. **Sort in Descending Order**:
$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$$
Sort eigenvectors accordingly.
8. **Select Principal Components**: Choose p such that:
$$\frac{\sum_{i=1}^{p} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \geq \text{threshold (e.g., 0.95)}$$
9. **Project Data**:
$$Y = X' \cdot E_p$$
where $E_p = [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_p]$

12 Interpretation of Results

12.1 Variance Explained

For our example with 4 features reduced to 2 principal components:

$$\text{Total Variance} = \sum_{i=1}^{4} \lambda_i = 2.2688 + 1.2064 + \lambda_3 + \lambda_4 = 4$$
$$\text{Variance Explained by PC1} = \frac{2.2688}{4} = 0.6189 = 61.89\%$$
$$\text{Variance Explained by PC2} = \frac{1.2064}{4} = 0.3288 = 32.88\%$$
$$\text{Cumulative Variance} = 61.89\% + 32.88\% = 94.77\%$$

12.2 Principal Components Interpretation

- **PC1 (61.89% variance)**:
  $\text{PC1} = 0.5472 \times \text{Feature1} + 0.5318 \times \text{Feature2} - 0.4792 \times \text{Feature3} - 0.4397 \times \text{Feature4}$
  Represents contrast between (Features 1, 2) and (Features 3, 4)
- **PC2 (32.88% variance)**:
  $\text{PC2} = 0.4199 \times \text{Feature1} + 0.1459 \times \text{Feature2} + 0.6446 \times \text{Feature3} + 0.6233 \times \text{Feature4}$
  Represents contrast between Feature1 and (Features 3, 4)

13 Applications & Practical Considerations

13.1 Applications in Machine Learning

- **Image Compression**: Eigenfaces in facial recognition
- **Gene Expression Analysis**: Identifying patterns in high-dimensional biological data
- **Finance**: Portfolio optimization and risk management
- **Signal Processing**: Noise reduction in audio signals
- **Natural Language Processing**: Document classification and topic modeling

13.2 Covariance vs Correlation Matrix

Aspect	Covariance Matrix	Correlation Matrix
Scale sensitivity	Sensitive	Insensitive
Feature scaling	Not required	Required (standardize)
Interpretation	Preserves units	Unitless
Variance range	Any	Diagonal = 1

Table 3: Comparison of covariance and correlation matrices

13.3 Choosing Number of Components

- **Kaiser Criterion**: Keep components with eigenvalue > 1
- **Scree Plot**: Look for "elbow" point
- **Variance Threshold**: Typically 90-95% cumulative variance
- **Cross-validation**: Based on model performance

14 Exercise Problems

14.1 Problem 1: Conceptual

Why must eigenvectors in PCA be orthogonal? What mathematical property ensures this?

14.2 Problem 2: Calculation

Given two features:

$$X = [2, 4, 6, 8, 10], \quad Y = [1, 3, 5, 7, 9]$$

Compute the covariance matrix and find the first principal component.

14.3 Problem 3: Eigenvalue Problem

For covariance matrix:

$$S = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

(a) Find eigenvalues and eigenvectors

(b) Calculate percentage of variance explained by each component

(c) What is the correlation between original features?

14.4 Problem 4: Python Implementation

Modify the from-scratch PCA function to:

- Accept an optional parameter for standardization
- Return the cumulative explained variance ratio
- Plot the scree plot automatically

14.5 Problem 5: Real-world Scenario

You have a dataset with 100 features. PCA gives eigenvalues:

$$[45.2, 18.7, 12.3, 8.9, 6.4, 3.2, 2.1, 1.8, 1.2, 0.9, \dots]$$

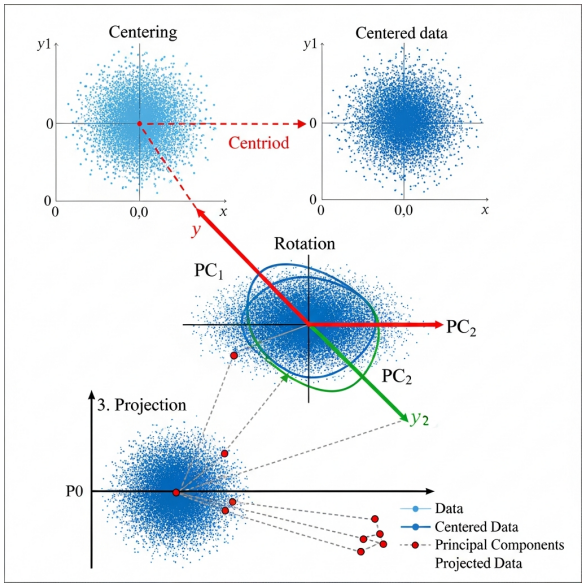
How many components would you choose and why? What would be the cumulative variance explained by the first 5 components?

14.6 Final Notes

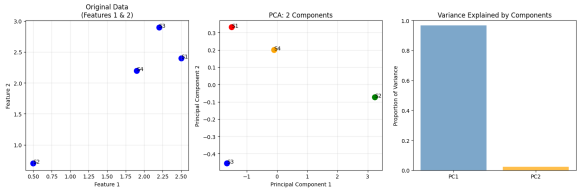
- PCA is **unsupervised** - doesn't use target labels
- Works best when data has **linear relationships**
- Consider **kernel PCA** for nonlinear data
- Always **validate** results with domain knowledge
- PCA reduces dimensions but **not necessarily** improves prediction

14.7 References and Further Reading

- Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer
- Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv:1404.1100
- Scikit-learn PCA Documentation

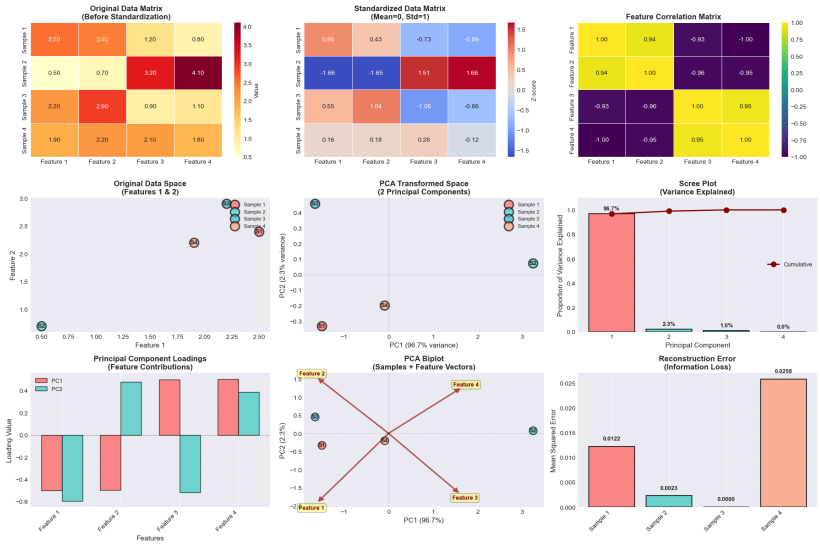


(a) Original Data (Features 1 & 2)



(b) PCA: 2 Components (94.8% variance explained)

Figure 4: Visualization comparing original features with PCA components



(a) PCA: 2 Components (94.8% variance explained)

Figure 5: Visualization comparing original features with PCA components